

深度学习

2025年9月

2.2 矩阵和向量相乘

2

```
torch.matmul(input, other, *, out=None)
tensor1 = torch.randn(4)
tensor2 = torch.randn(10, 4, 3)
print(torch.matmul(tensor1, tensor2).size())
```

```
tensor1 = torch.randn(4)
tensor2 = torch.randn(10, 2, 4, 3)
print(torch.matmul(tensor1, tensor2).size())
```

```
>>> tensor1 = torch.randn(10, 3, 4)
>>> tensor2 = torch.randn(10, 4, 5)
>>> torch.matmul(tensor1, tensor2).size()
torch.Size([10, 3, 5])
```



2 数值回顾

3

arange

首先，我们可以使用 `arange` 创建一个行向量 `x`。这个行向量包含以0开始的前12个整数，它们默认创建为整数。也可指定创建类型为浮点数。张量中的每个值都称为张量的 元素（element）。例如，张量 `x` 中有 12 个元素。除非额外指定，新的张量将存储在内存中，并采用基于CPU的计算。

```
x = torch.arange(12)  
x
```

```
tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

2 数值回顾

4

shape

可以通过张量的shape属性来访问张量（沿每个轴的长度）的形状。

MXNET

PYTORCH

TENSORFLOW

PADDLE

`x.shape`



```
torch.Size([12])
```

2 数值回顾

5

reshape

要想改变一个张量的形状而不改变元素数量和元素值，可以调用reshape函数。例如，可以把张量x从形状为(12,)的行向量转换为形状为(3,4)的矩阵。这个新的张量包含与转换前相同的值，但是它被看成一个3行4列的矩阵。要重点说明一下，虽然张量的形状发生了改变，但其元素值并没有变。注意，通过改变张量的形状，张量的大小不会改变。

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
X = x.reshape(3, 4)
X
```



```
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

我们不需要通过手动指定每个维度来改变形状。也就是说，如果我们的目标形状是(高度,宽度)，那么在知道宽度后，高度会被自动计算得出，不必我们自己做除法。在上面的例子中，为了获得一个3行的矩阵，我们手动指定了它有3行和4列。幸运的是，我们可以通过-1来调用此自动计算出维度的功能。即我们可以用x.reshape(-1,4)或x.reshape(3,-1)来取代x.reshape(3,4)。

2 数值回顾

6

有时，我们希望使用全0、全1、其他常量，或者从特定分布中随机采样的数字来初始化矩阵。我们可以创建一个形状为(2,3,4)的张量，其中所有元素都设置为0。代码如下：

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
torch.zeros((2, 3, 4))
```



```
tensor([[[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]],
        [[0., 0., 0., 0.],
         [0., 0., 0., 0.],
         [0., 0., 0., 0.]])
```

2 数值回顾

7

同样，我们可以创建一个形状为(2,3,4)的张量，其中所有元素都设置为1。代码如下：

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
torch.ones((2, 3, 4))
```



```
tensor([[[1., 1., 1., 1.],  
         [1., 1., 1., 1.],  
         [1., 1., 1., 1.]],  
        [[1., 1., 1., 1.],  
         [1., 1., 1., 1.],  
         [1., 1., 1., 1.]])
```

2 数值回顾

8

有时我们想通过从某个特定的概率分布中随机采样来得到张量中每个元素的值。例如，当我们构造数组来作为神经网络中的参数时，我们通常会随机初始化参数的值。以下代码创建一个形状为 (3,4) 的张量。其中的每个元素都从均值为 0、标准差为 1 的标准高斯分布（正态分布）中随机采样。

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
torch.randn(3, 4)
```



```
tensor([[ -0.0135,  0.0665,  0.0912,  0.3212],  
        [ 1.4653,  0.1843, -1.6995, -0.3036],  
        [ 1.7646,  1.0450,  0.2457, -0.7732]])
```

2 数值回顾

9

我们也可以把多个张量连结 (concatenate) 在一起, 把它们端对端地叠起来形成一个更大的张量。我们只需要提供张量列表, 并给出沿哪个轴连结。 下面的例子分别演示了当我们沿行 (轴-0, 形状的第一个元素) 和按列 (轴-1, 形状的第二个元素) 连结两个矩阵时, 会发生什么情况。我们可以看到, 第一个输出张量的轴-0长度 (6) 是两个输入张量轴-0长度的总和 ($3 + 3$); 第二个输出张量的轴-1长度 (8) 是两个输入张量轴-1长度的总和 ($4 + 4$)。

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
X = torch.arange(12, dtype=torch.float32).reshape((3,4))
Y = torch.tensor([[2.0, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])
torch.cat((X, Y), dim=0), torch.cat((X, Y), dim=1)
```



```
(tensor([[ 0.,  1.,  2.,  3.],
         [ 4.,  5.,  6.,  7.],
         [ 8.,  9., 10., 11.],
         [ 2.,  1.,  4.,  3.],
         [ 1.,  2.,  3.,  4.],
         [ 4.,  3.,  2.,  1.])),
 tensor([[ 0.,  1.,  2.,  3.,  2.,  1.,  4.,  3.],
         [ 4.,  5.,  6.,  7.,  1.,  2.,  3.,  4.],
         [ 8.,  9., 10., 11.,  4.,  3.,  2.,  1.])))
```

cat

在大多数情况下，我们将沿着数组中长度为1的轴进行广播，如下例子：

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
a = torch.arange(3).reshape((3, 1))
b = torch.arange(2).reshape((1, 2))
a, b
```

```
(tensor([[0],
         [1],
         [2]]),
 tensor([[0, 1]]))
```

由于a和b分别是 3×1 和 1×2 矩阵，如果让它们相加，它们的形状不匹配。我们将两个矩阵广播为一个更大的 3×2 矩阵，如下所示：矩阵a将复制列，矩阵b将复制行，然后再按元素相加。

MXNET

PYTORCH

TENSORFLOW

PADDLE

a + b

a 广播后：

```
[[0, 0]
 [1, 1]
 [2, 2]]
```

b 广播后：

```
[[0, 1]
 [0, 1]
 [0, 1]]
```

```
tensor([[0, 1],
        [1, 2],
        [2, 3]])
```

就像在任何其他Python数组中一样，张量中的元素可以通过索引访问。与任何Python数组一样：第一个元素的索引是0，最后一个元素索引是-1；可以指定范围以包含第一个元素和最后一个之前的元素。

11

如下所示，我们可以用[-1]选择最后一个元素，可以用[1:3]选择第二个和第三个元素：

MXNET

PYTORCH

TENSORFLOW

PADDLE

`X[-1], X[1:3]`

索引和切片

```
(tensor([ 8.,  9., 10., 11.]),
 tensor([[ 4.,  5.,  6.,  7.],
         [ 8.,  9., 10., 11.])))
```

除读取外，我们还可以通过指定索引来将元素写入矩阵。

```
X[1, 2] = 9
X
```

```
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  9.,  7.],
        [ 8.,  9., 10., 11.]])
```

```
compute_result.py x
1 import numpy as np
2 import torch
3
4 x = torch.arange(12).reshape(*shape: 3, 4)
5 print(x)
6 print("oooooooooooooooooooooooooooo")
7 print(x[-1])
8 print("#####")
9 print(x[1:3])

compute_result x
:
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myv
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
oooooooooooooooooooooooooooo
tensor([ 8,  9, 10, 11])
#####
tensor([[ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])

Process finished with exit code 0
```

2 数值回顾

12

如果我们想为多个元素赋值相同的值，我们只需要索引所有元素，然后为它们赋值。例如，`[0:2, :]`访问第1行和第2行，其中“:”代表沿轴1（列）的所有元素。虽然我们讨论的是矩阵的索引，但这也适用于向量和超过2个维度的张量。

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
X[0:2, :] = 12  
X
```



```
tensor([[12., 12., 12., 12.],  
        [12., 12., 12., 12.],  
        [ 8.,  9., 10., 11.]])
```

2.3.1 创建数据集

13

举一个例子，我们首先创建一个人工数据集，并存储在CSV（逗号分隔值）文件 `../data/house_tiny.csv` 中。以其他格式存储的数据也可以通过类似的方式进行处理。下面我们将数据集按行写入CSV文件中。

```
import os

os.makedirs(os.path.join '..', 'data'), exist_ok=True)
data_file = os.path.join '..', 'data', 'house_tiny.csv')
with open(data_file, 'w') as f:
    f.write('NumRooms,Alley,Price\n') # 列名
    f.write('NA,Pave,127500\n') # 每行表示一个数据样本
    f.write('2,NA,106000\n')
    f.write('4,NA,178100\n')
    f.write('NA,NA,140000\n')
```

首先使用 `os.makedirs()` 创建了一个文件夹 `../data`，然后创建了一个 CSV 文件 `house_tiny.csv`，并写入数据。文件的表头定义了三列 `NumRooms`（房间数）、`Alley`（道路类型）、`Price`（房价），后面的四行是房屋的具体数据样本，其中有部分数据缺失（用 `NA` 表示）。

2.3.2 读取数据集

14

```
# 如果没有安装pandas, 只需取消对以下行的注释来安装pandas
# !pip install pandas
import pandas as pd

data = pd.read_csv(data_file)
print(data)
```

	NumRooms	Alley	Price
0	NaN	Pave	127500
1	2.0	NaN	106000
2	4.0	NaN	178100
3	NaN	NaN	140000

2.3.3 数据预处理

15

通过位置索引iloc，我们将data分成inputs和outputs，其中前者为data的前两列，而后者为data的最后一列。对于inputs中缺少的数值，我们用同一列的均值替换“NaN”项。

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
inputs, outputs = data.iloc[:, 0:2], data.iloc[:, 2]
inputs = inputs.fillna(inputs.mean())
print(inputs)
```

分成inputs和outputs

	NumRooms	Alley
0	3.0	Pave
1	2.0	NaN
2	4.0	NaN
3	3.0	NaN

	NumRooms	Alley
0	NaN	Pave
1	2.0	NaN
2	4.0	NaN
3	NaN	NaN

2.3.4 数据预处理

16

对于inputs中的类别值或离散值，我们将“NaN”视为一个类别。由于“巷子类型”（“Alley”）列只接受两种类型的类别值“Pave”和“NaN”，pandas可以自动将此列转换为两列“Alley_Pave”和“Alley_nan”。巷子类型为“Pave”的行会将“Alley_Pave”的值设置为1，“Alley_nan”的值设置为0。缺少巷子类型的行会将“Alley_Pave”和“Alley_nan”分别设置为0和1。

MXNET

PYTORCH

TENSORFLOW

PADDLE

```
inputs = pd.get_dummies(inputs, dummy_na=True)
print(inputs)
```

	NumRooms	Alley_Pave	Alley_nan
0	3.0	1	0
1	2.0	0	1
2	4.0	0	1
3	3.0	0	1

	NumRooms	Alley
0	3.0	Pave
1	2.0	NaN
2	4.0	NaN
3	3.0	NaN

2.3.5 转化为张量

17

```
import torch
```

```
X = torch.tensor(inputs.to_numpy(dtype=float))  
y = torch.tensor(outputs.to_numpy(dtype=float))  
X, y
```

```
(tensor([[3., 1., 0.],  
        [2., 0., 1.],  
        [4., 0., 1.],  
        [3., 0., 1.]], dtype=torch.float64),  
 tensor([127500., 106000., 178100., 140000.], dtype=torch.float64))
```

第三章 概率与信息论

18



3.1 为什么要使用概率

3.2 随机变量

3.3 概率分布

3.4 边缘概率

3.5 条件概率



3.6 条件概率的链式法则

3.7 独立性和条件独立性

3.8 期望、方差和协方差

3.9 常用概率分布

3.10 深度学习相关案例

第三章 概率与信息论

20

现在让我们更认真地考虑第一个例子：根据照片区分猫和狗。这听起来可能很简单，但对于机器却可能是一个艰巨的挑战。首先，问题的难度可能取决于图像的分辨率。



图2.6.1 不同分辨率的图像 (10×10 , 20×20 , 40×40 , 80×80 , 和 160×160 pixels)

3.1 为什么要使用概率

21

- 不确定性：不确定性指事先不能准确地知道某种决策的结果。或者说，可能的结果不止一种。
- 随机性：具有某一概率的事件集合中的各个事件所表现出来的不确定性。
- 机器学习通常必须处理**不确定量**，有时也可能需要处理**随机量**。不确定性和随机性可能来自多个方面。

3.1 为什么要使用概率

22

- 扔一颗骰子，结果只能是六面之一，且每一面出现的可能性都是 $1/6$
- 通过代码生成一个随机数，设定边界参数，可能出现的数字及每个数字出现的概率就已确定
- 买了一张福利彩票，最后的结果可能是不中奖、一等奖...六等奖，共七种可能性，每种可能性出现的概率都是可提前计算的

3.1 为什么要使用概率

23

- 《反脆弱》中讲了这么一则故事，假设你是一只火鸡，你在来到人类世界的第一天，一个屠夫出现在你的生活中。你担心他会杀了你，但他很和善，还喂给你食物。第二天、第三天、第四天同样如此，这个屠夫会一直继续喂你食物吗？这个概率可以计算吗？

3.1 为什么要使用概率

24

避免因为未观测到的事件而导致零概率问题，确保模型在面对稀疏数据时仍然具有一定的泛化能力

- 著名数学家拉普拉斯在研究太阳升起问题时提出了拉普拉斯平滑理论
- 如果在第 $n+1$ 天，我们观察到之前太阳升起了 s 次，则今天太阳升起的概率为

$$P(\text{太阳升起}|\text{之前的观测}) = \frac{s + 1}{n + 2}$$

3.1 为什么要使用概率

25

- 但不幸的是，第100天正好是感恩节，根据你的计算被杀的可能性比之前任何一天都小，但你却在这一天大难临头了。
- 这里的关键是，感恩节对于你来说是**未知信息**，如果你提前知道所有可能的风险，你很可能会重新计算每天被杀的概率。

3.1 为什么要使用概率

26

- 在扑克牌游戏中抽出一手特定的牌，事件是可重复的
- 一个医生诊断病人患流感的几率为 40%，病人没有副本，事件是不可重复的
- 可重复事件的概率，直接与事件发生的频率相联系，被称为**频率派概率** (frequentist probability)
- 在医生诊断病人的例子中，我们用概率来表示一种信任度 (degree of belief)，其中 1 表示非常肯定病人患有流感，0 表示非常肯定病人没有流感。通过概率来表示一种信任度，设计确定性水平，被称为**贝叶斯概率** (Bayesian probability) 。

3.1 为什么要使用概率

27

- 从盒子中抽取小球的经典问题，可以通过多次实验的方式来计算频率，并进而估算概率
- 南极大陆在本世纪末完全融化的可能性，这类事件其不可能被重复出现，无法使用频率统计的思想来进行概率的估计
- 在贝叶斯方法中，我们关心的是通过现有数据来更新对某个事件或参数的信念。这涉及到先验概率和后验概率的计算。

关于 w 的似然函数 $p(D|w)$ 、关于 w 的先验概率

$$p(w|D) = \frac{p(D|w)p(w)}{p(D)}$$

参数 w 的不确定度

对数据集 D 的观测

3.1 为什么要使用概率

28

先验概率 $p(w)$: 是我们在观察到数据之前，对参数 w 的初始信念。例如，关于南极洲是否会融化的先验知识可能基于科学研究、过去气候变化的模式等。

似然函数 $p(D|w)$: 表示在给定参数 w 的情况下，观测到数据 D 的可能性。它描述了模型参数如何影响观察结果。例如，假设科学家通过气候模型得到了在某个温度升高条件下南极融化的概率。

后验概率 $p(w|D)$: 是我们在观察到数据之后，更新对参数 w 的信念。通过**贝叶斯定理**，我们可以将先验概率和似然函数结合起来，得到后验概率：

$$p(w|D) = \frac{p(D|w) \cdot p(w)}{p(D)}$$

假设科学家有气候变化模型和历史观测数据（数据集 D ），那么就可以通过这些模型和数据估计出未来南极大陆融化的可能性

3.1 为什么要使用概率

29

- 贝叶斯派中的一个优势在于对于先验知识的包含是逐步上升的。比如，抛硬币抛出10次正面，频率学派依据似然函数认为抛出正面的概率为1，贝叶斯派则会考虑先验信息，现实中这种情况出现的概率太低了。因此可以中和频率学派的极端假设。
- 然而，对于贝叶斯派来说，一个普遍的批评在于该方法选择的先验知识可能并非是一个独立的事件，因此，**基于差的先验选择可能会给差的结果一个较高的可信度**。
- 但频率派方法则不存在该问题，因为频率派可以使用**交叉验证**的方法减少因为选择先验（训练集）而对于模型的影响。

3.1 为什么要使用概率

30

- 频率派认为模型的参数 θ 是一个未知的常量，数据 X 是随机变量，关心的是数据，需要把参数 θ 估计出来，常用的方法就是极大似然估计。

$$\theta = \arg \max \log p(x | \theta)$$

- 贝叶斯派认为 θ 是一个随机变量，服从一个概率分布 $\theta \sim p(\theta)$ ，称之为先验概率。通过贝叶斯定理把参数的先验和后验以及似然联系起来。

$$p(\theta|x) = \frac{p(x|\theta)p(\theta)}{p(x)}$$

3.2 随机变量

31

- 随机变量作为一种映射后的取值，本质上和函数取值一样，可以有连续型和离散型两种
- 离散型随机变量：随机变量的取值只能是有限多个或者是可数的无穷多个值
- 对于向量值变量，我们会将随机变量写成 $\mathbf{x}$ ，它的一个可能取值为 $\mathbf{x}$ 。

3.2 随机变量

32

- 随机变量的要素：
 - 第一： 随机变量的取值。 由试验结果衍生出的这一组随机变量到底能取到哪些值
 - 第二： 试验中每个对应取值的概率。
 - 第三： 随机变量的统计特征和度量方法。

3.3 概率分布

33

- 概率分布 (probability distribution) 用来描述随机变量或一族随机变量在每一个可能取到的状态的可能性大小。我们描述概率分布的方式取决于随机变量是离散的还是连续的。

3.3.1 离散型变量和概率质量函数

34

- 分布列描述的是离散型随机变量每一种取值及其对应的概率

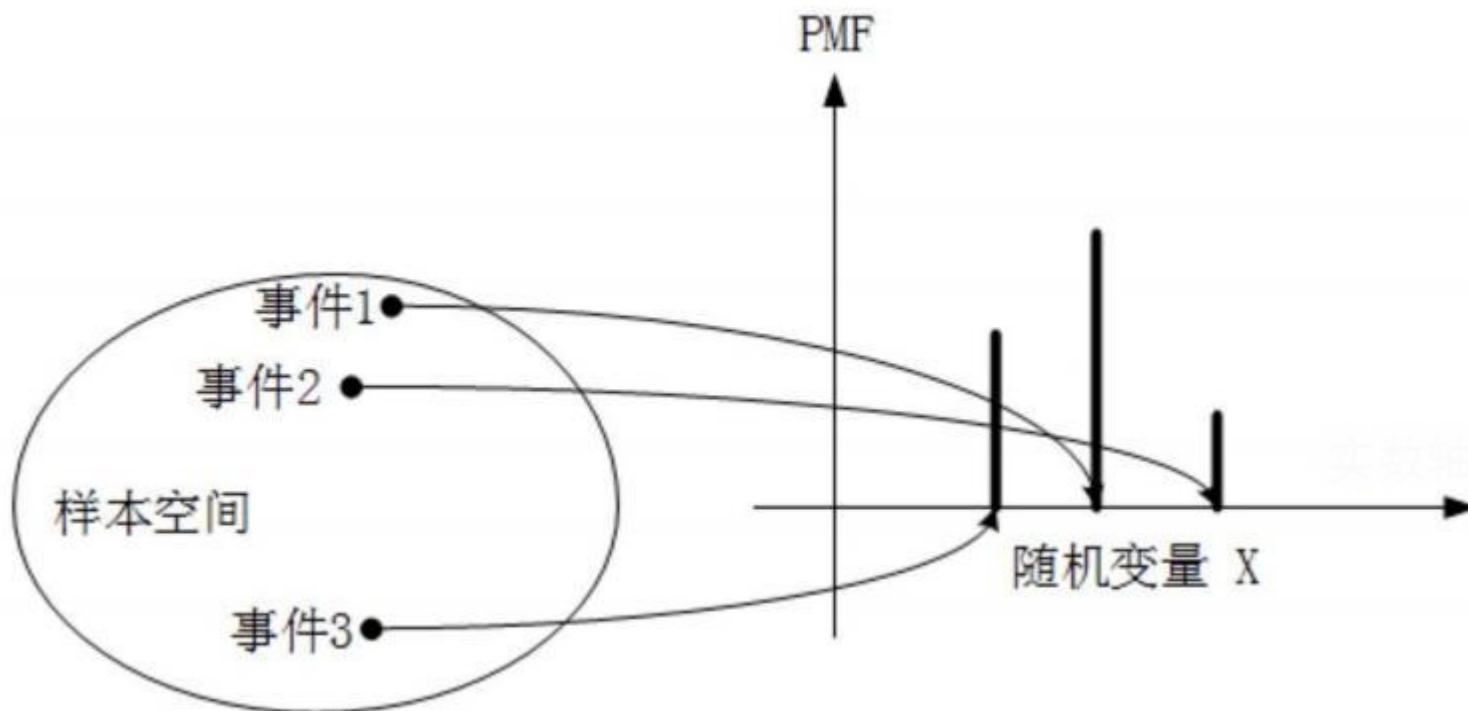
随机变量 X 取值为 x 的概率，本质上也是一个事件的概率，这个事件就是 $\{X = x\}$ ，我们将他记作： $P_X(x) = P(\{X = x\})$ 。

取值	0	1	2	其他
P	$\frac{1}{4}$	$\frac{1}{2}$	$\frac{1}{4}$	0

3.3.1 离散型变量和概率质量函数

35

- 离散型变量的概率分布可以用**概率质量函数**（probability mass function, PMF）来描述。我们通常用大写字母 P 来表示概率质量函数。通常每一个随机变量都会有一个不同的概率质量函数
- 概率质量函数将随机变量能够取得的每个状态映射到随机变量取得该状态的概率。



3.3.1 离散型变量和概率质量函数

36

概率质量函数将随机变量能够取得的每个状态映射到随机变量取得该状态的概率。 $x = x$ 的概率用 $P(x)$ 来表示，概率为 1 表示 $x = x$ 是确定的，概率为 0 表示 $x = x$ 是不可能发生的。有时为了使得PMF的使用不相互混淆，我们会明确写出随机变量的名称： $P(x = x)$ 。有时我们会先定义一个随机变量，然后用 $\sim$ 符号来说明它遵循的分布： $x \sim P(x)$ 。

3.3.1 离散型变量和概率质量函数

37

概率质量函数可以同时作用于多个随机变量。这种多个变量的概率分布被称为联合概率分布 (joint probability distribution)。 $P(x = x, y = y)$ 表示 $x = x$ 和 $y = y$ 同时发生的概率。我们也可以简写为 $P(x, y)$ 。

3.3.1 离散型变量和概率质量函数

38

- 将一个硬币抛掷 n 次，每次抛掷出现正面的概率为 p ，每次抛掷彼此之间都是相互独立的，随机变量 X 对应 “ n 次抛掷得到的是正面” 的次数

$$p_X(k) = P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

3.3.2 连续型变量和概率密度函数

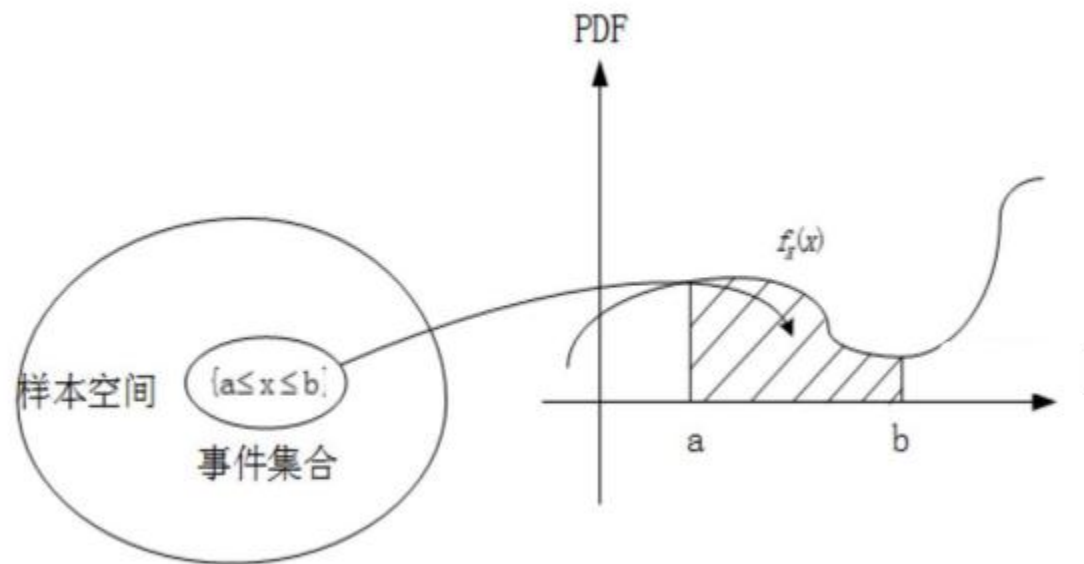
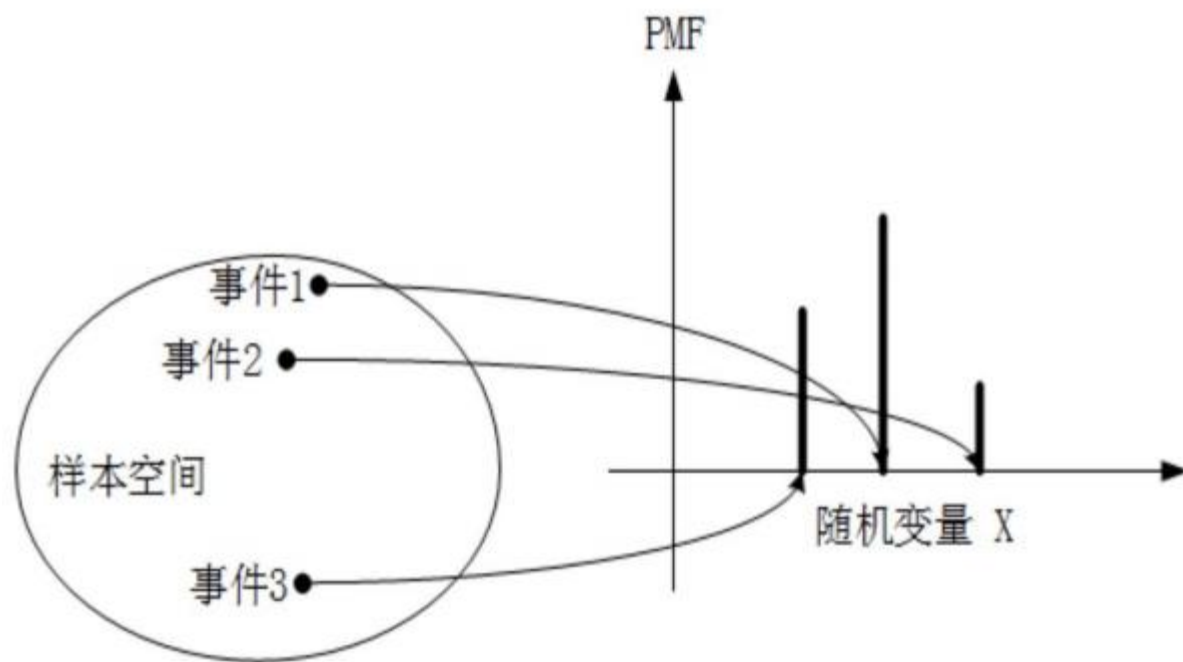
39

当我们研究的对象是连续型随机变量时，我们用概率密度函数（probability density function, PDF）而不是概率质量函数来描述它的概率分布。如果一个函数 p 是概率密度函数，必须满足下面这几个条件：

- p 的定义域必须是 x 所有可能状态的集合。
- $\forall x \in x, p(x) \geq 0$. 注意，我们并不要求 $p(x) \leq 1$ 。
- $\int p(x)dx = 1$.

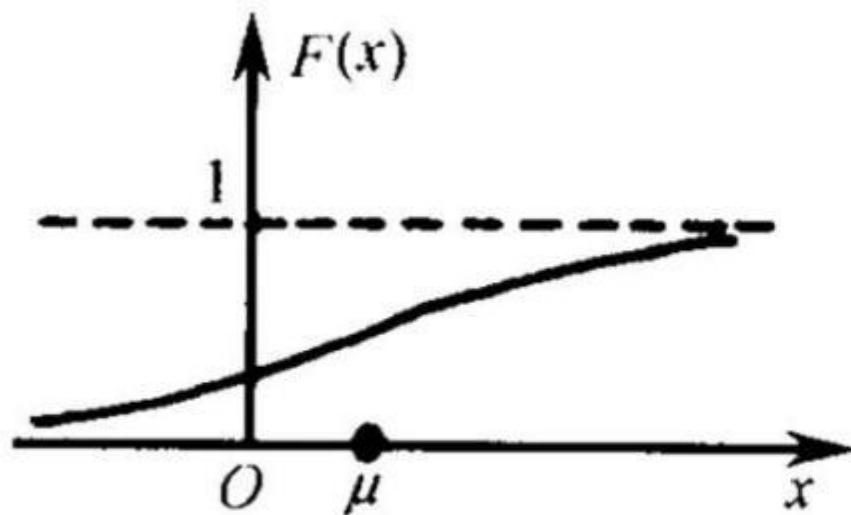
3.3.2 连续型变量和概率密度函数

40

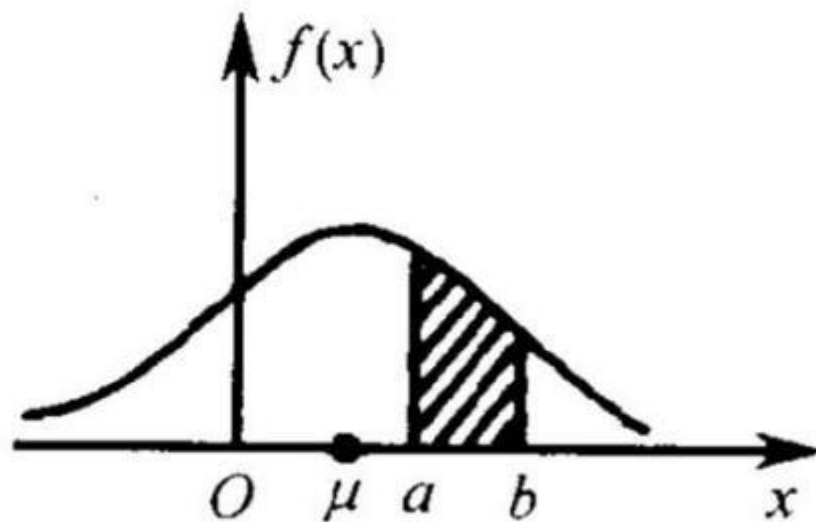


3.3.2 连续型变量和概率密度函数

41



(a)



(b)

<https://blog.csdn.net/Anne033>

3.4 边缘概率

42

$P(X = a)$ 或 $P(Y = b)$ 这类仅与单个随机变量有关的概率，称为边缘概率。

$$P(X = a) = \sum_b P(X = a, Y = b)$$

$$P(Y = b) = \sum_a P(X = a, Y = b)$$

求和符号 $\sum_b (\dots)$ 表示“穷举 Y 可取的值 b 后，由所有与这些值对应的 $(\dots)$ 相加得到的和”。

类似地， $\sum_a (\dots)$ 表示“穷举 X 可取的值 a 后，由所有与这些值对应的 $(\dots)$ 相加得到的和”。

<https://blog.csdn.net/YPPO>

3.5 条件概率

43

在很多情况下，我们感兴趣的是某个事件，在给定其他事件发生时出现的概率。这种概率叫做条件概率。我们将给定 $x = x$, $y = y$ 发生的条件概率记为 $P(y = y \mid x = x)$ 。这个条件概率可以通过下面的公式计算：

$$P(y = y \mid x = x) = \frac{P(y = y, x = x)}{P(x = x)}. \quad (3.5)$$

3.6 条件概率的链式法则

44

任何多维随机变量的联合概率分布，都可以分解成只有一个变量的条件概率相乘的形式：

$$P(x^{(1)}, \dots, x^{(n)}) = P(x^{(1)}) \prod_{i=2}^n P(x^{(i)} \mid x^{(1)}, \dots, x^{(i-1)}). \quad (3.6)$$

这个规则被称为概率的 **链式法则**（chain rule）或者 **乘法法则**（product rule）。它可以直接从式 (3.5) 条件概率的定义中得到。例如，使用两次定义可以得到

$$P(a, b, c) = P(a \mid b, c)P(b, c)$$

$$P(b, c) = P(b \mid c)P(c)$$

$$P(a, b, c) = P(a \mid b, c)P(b \mid c)P(c).$$

将计算多个事件的联合概率问题分解成一系列条件概率问题

3.7 独立性和条件独立性

45

两个随机变量 x 和 y , 如果它们的概率分布可以表示成两个因子的乘积形式, 并且一个因子只包含 x 另一个因子只包含 y , 我们就称这两个随机变量是相互独立的 (independent):

$$\forall x \in \mathbf{x}, y \in \mathbf{y}, p(x = x, y = y) = p(x = x)p(y = y). \quad (3.7)$$

如果关于 x 和 y 的条件概率分布对于 z 的每一个值都可以写成乘积的形式, 那么这两个随机变量 x 和 y 在给定随机变量 z 时是条件独立的

$$\forall x \in \mathbf{x}, y \in \mathbf{y}, z \in \mathbf{z}, p(x = x, y = y \mid z = z) = p(x = x \mid z = z)p(y = y \mid z = z). \quad (3.8)$$

3.8 期望、方差和协方差

46

函数 $f(x)$ 关于某分布 $P(x)$ 的期望 (expectation) 或者期望值 (expected value) 是指, 当 x 由 P 产生, f 作用于 x 时, $f(x)$ 的平均值。对于离散型随机变量, 这可以通过求和得到:

$$\mathbb{E}_{x \sim P}[f(x)] = \sum_x P(x) f(x), \quad (3.9)$$

对于连续型随机变量可以通过求积分得到:

$$\mathbb{E}_{x \sim p}[f(x)] = \int p(x) f(x) dx. \quad (3.10)$$

3.8 期望、方差和协方差

47

当概率分布在上下文中指明时，我们可以只写出期望作用的随机变量的名称来进行简化，例如 $\mathbb{E}_x[f(x)]$ 。如果期望作用的随机变量也很明确，我们可以完全不写脚标，就像 $\mathbb{E}[f(x)]$ 。默认地，我们假设 $\mathbb{E}[\cdot]$ 表示对方括号内的所有随机变量的值求平均。类似的，当没有歧义时，我们还可以省略方括号。

期望是线性的，例如，

$$\mathbb{E}_x[\alpha f(x) + \beta g(x)] = \alpha \mathbb{E}_x[f(x)] + \beta \mathbb{E}_x[g(x)], \quad (3.11)$$

其中 α 和 β 不依赖于 x 。

3.8 期望、方差和协方差

48

方差（variance）衡量的是当我们对 x 依据它的概率分布进行采样时，随机变量 x 的函数值会呈现多大的差异：

$$\text{Var}(f(x)) = \mathbb{E} [(f(x) - \mathbb{E}[f(x)])^2] . \quad (3.12)$$

当方差很小时， $f(x)$ 的值形成的簇比较接近它们的期望值。方差的平方根被称为**标准差**（standard deviation）。

3.8 期望、方差和协方差

49

协方差 (covariance) 在某种意义上给出了两个变量线性相关性的强度以及这些变量的尺度：

$$\text{Cov}(f(x), g(y)) = \mathbb{E}[(f(x) - \mathbb{E}[f(x)])(g(y) - \mathbb{E}[g(y)])]. \quad (3.13)$$

协方差的绝对值如果很大则意味着变量值变化很大并且它们同时距离各自的均值很远。如果协方差是正的，那么两个变量都倾向于同时取得相对较大的值。如果协方差是负的，那么其中一个变量倾向于取得相对较大的值的同时，另一个变量倾向于取得相对较小的值，反之亦然。其他的衡量指标如 **相关系数** (correlation) 将每个变量的贡献归一化，为了只衡量变量的相关性而不受各个变量尺度大小的影响。

3.8 期望、方差和协方差

50

随机向量 $\mathbf{x} \in \mathbb{R}^n$ 的协方差矩阵 (covariance matrix) 是一个 $n \times n$ 的矩阵, 并且满足

$$\text{Cov}(\mathbf{x})_{i,j} = \text{Cov}(x_i, x_j). \quad (3.14)$$

协方差矩阵的对角元是方差:

$$\text{Cov}(x_i, x_i) = \text{Var}(x_i). \quad (3.15)$$

3.9 常用概率分布

51

3.9.1 Bernoulli 分布

3.9.2 Multinoulli分布 范畴分布

3.9.3 高斯分布

3.9.1 Bernoulli 分布

52

Bernoulli 分布 (Bernoulli distribution) 是单个二值随机变量的分布。它由单个参数 $\phi \in [0, 1]$ 控制, ϕ 给出了随机变量等于 1 的概率。它具有如下的一些性质:

$$P(x = 1) = \phi \quad (3.16)$$

$$P(x = 0) = 1 - \phi \quad (3.17)$$

$$P(x = x) = \phi^x (1 - \phi)^{1-x} \quad (3.18)$$

$$\mathbb{E}_x[x] = \phi \quad (3.19)$$

$$\text{Var}_x(x) = \phi(1 - \phi) \quad (3.20)$$

3.9.2 Multinoulli分布

53

Multinoulli 分布 (multinoulli distribution) 或者 **范畴分布** (categorical distribution) 是指在具有 k 个不同状态的单个离散型随机变量上的分布, 其中 k 是一个有限值。² Multinoulli 分布由向量 $\mathbf{p} \in [0, 1]^{k-1}$ 参数化, 其中每一个分量 p_i 表示第 i 个状态的概率。最后的第 k 个状态的概率可以通过 $1 - \mathbf{1}^\top \mathbf{p}$ 给出。注意我们必须限制 $\mathbf{1}^\top \mathbf{p} \leq 1$ 。Multinoulli 分布经常用来表示对象分类的分布, 所以我们很少假设状态 1 具有数值 1 之类的。因此, 我们通常不需要去计算 Multinoulli 分布的随机变量的期望和方差。

3.9.3 高斯分布

54

实数上最常用的分布就是正态分布（normal distribution），也称为高斯分布

$$\mathcal{N}(x; \mu, \sigma^2) = \sqrt{\frac{1}{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right).$$

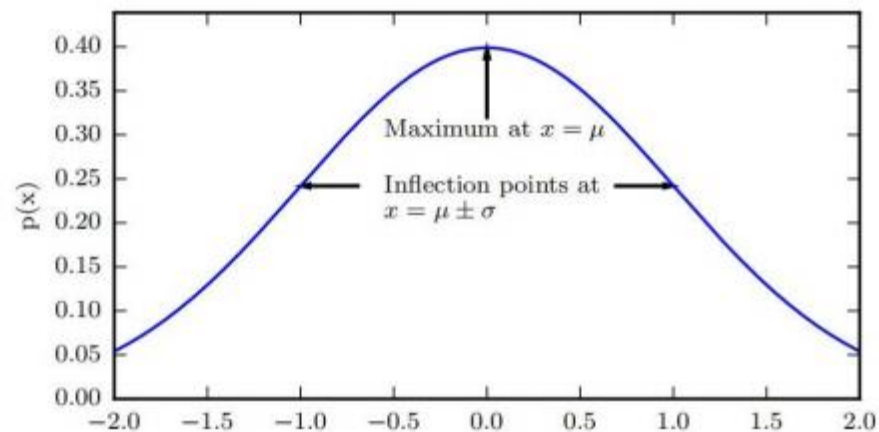


图 3.1: 正态分布。正态分布 $\mathcal{N}(x; \mu, \sigma^2)$ 呈现经典的“钟形曲线”的形状，其中中心峰的 x 坐标由 μ 给出，峰的宽度受 σ 控制。在这个示例中，我们展示的是标准正态分布（standard normal distribution），其中 $\mu = 0, \sigma = 1$ 。